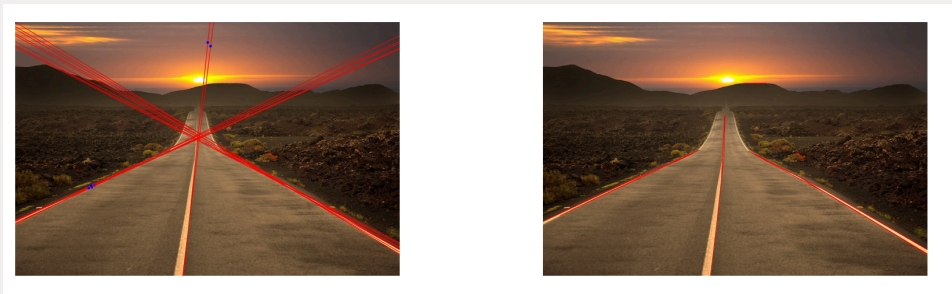


정리_컴퓨터비전

- 각 개념 정리 및 PiLLOW & OpenCV에서의 전처리 적용
- **PyTorch를 활용한 데이터 증강 예제코드의 transform V1을 V2로 변경**
- 가장하단의 4.2 실습: **PyTorch를 활용한 이미지 처리 파이프라인 의 로더기의 옵션값 정리**

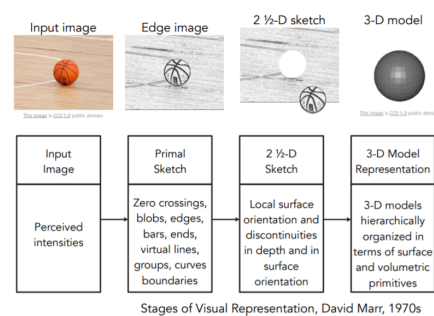
1 컴퓨터비전

- 컴퓨터 비전은 인간의 시각적 지능을 모방하려는 노력에서 시작
- 다양한 기술 발전과 함께 현대의 딥러닝 기반 비전 기술로 발전
- **초기목표** : 단순한 패턴 인식과 경계 검출
- **Hubel & Wiesel (1959)**
 - 영장류와 포유류가 어떤 메커니즘으로 이미지를 처리하는지 연구
 - 고양이의 시각을 담당하는 뉴런들이 Oriented edge가 어떤 방향으로 이동할때 반응한다는 사실을 확인
 - 시각적 처리가 단순한 구조에서 시작된다는 것을 의미
 - 뇌는 복잡한 정보를 인식할 수 있을때 까지 시각정보의 복잡성을 증가
- **Hough Transform (1962)**
 - 직선, 원 등 기하학적 형태의 탐지를 위한 알고리즘
 - 직선 검출을 위한 가장 기본이 되었던 알고리즘 → 차선 검출 등에 사용



• Marr의 "Vision" 이론 (1982)

- 시각을 저수준, 중간수준, 고수준 처리 단계로 분리.
 - 저수준 처리 (Low-Level Vision):
경계 검출, 텍스처 분석 등 기본적인 시각 처리.
 - 중간수준 처리 (Mid-Level Vision):
객체를 세그멘테이션하거나, 경계선을 연결하여 더 큰 구조를 식별.
 - 고수준 처리 (High-Level Vision):
전체 장면을 이해하거나, 객체를 인식하는 단계.
- 우리가 눈으로 받아들인 이미지를 최종적인 full 3D 표현으로 만들려면 몇 단계의 과정을 거쳐야 한다고 주장



• Primal Sketch :

- 경계(edges), 막대(bars), 끝(ends), virtual lines, curves, boundaries가 표현
- 이전에 Hubel과 Wiesel은 시각 처리의 초기 단계는 경계(edges)와 같은 단순한 구조와 밀접한 관련이 있다고 주장했음

• 2.5D Sketch :

- 표면(surfaces), 깊이(depth), 층(layer), 불연속 점과 같은 것들을 종합하여 표현

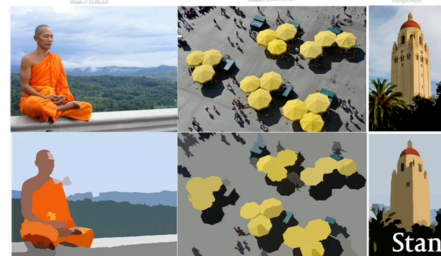
• 3D Model :

- 앞의 단계들을 모두 모아 surface and volumetric primitives 형태의 계층적으로 조직화된 최종적인 3D 모델

• Image Segmentation - Normalized Cut (1997)

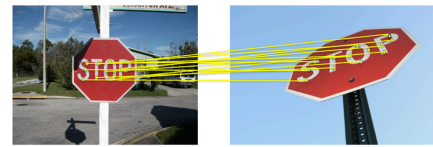
- 물체 인식 알고리즘 실현이 너무 어려움
→ 이미지 세그멘테이션부터 해보자
- 지금은 Image Segmentation이 detection 이나 Classification 보다는 어렵다고 보일수 있음
→ 분석이나 검출을 하려면 물체의 맥락을 이해해야 하므로 어려운 문제
→ Segmentation은 맥락 파악 보다는 주변 픽셀과의 관계를 이용해 수행할 수 있음

Normalized Cut (Shi & Malik, 1997)



• SIFT(Scale-Invariant Feature Transform)

- 다양한 크기와 회전에 강인한 특징 추출 기법
- 90년대 후반부터 2000년대까지 가장 영향력 있던 이미지 인식 기법
- 두개의 물체를 매칭
→ 카메라 각도, 조명, 크기 변화 등으로 인해 어려움
→ 일부 특징들은 이미지의 다양한 변화에서도 변화가 적음.
- Object 인식등에 사용됨



"SIFT" & Object Recognition, David Lowe, 1999

• HOG(Histogram of Oriented Gradients)

- 이미지의 경계와 방향 패턴을 기반으로 객체 감지
- 이미지 특정 영역에서 Edge의 크기와 방향을 추출
→ 저차원의 특징들을 수집해 고차원으로 맵핑
- SVM, boost 알고리즘등과 결합하여 보행자 검출이나 얼굴 검출 등에 활용

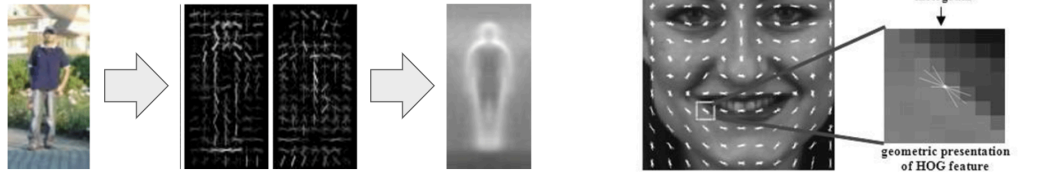


fig. 3. Histogram of oriented gradient extraction from face

• ImageNet 프로젝트의 시작 (2009)

- 대규모 데이터셋을 구축 (약 1400만 이미지)
- ImageNet 대회(ILSVRC, ImageNet Large Scale Visual Recognition Challenge)는 객체 탐지와 분류 성능을 벤치마크하는 표준이 됨
- 1000 class로 분류

• AlexNet (2012)

- ImageNet 대회에서 우승하며 딥러닝 성능을 입증
- 주요 특징
 - Convolution Neural Network (CNN)의 활용
 - ReLU 활성화 함수와 Dropout 기법으로 학습 효율 향상
 - GPU를 활용한 병렬 연산

• 이후 모델의 발전

- VGGNet (2014) : 단순하지만 매우 깊은 네트워크 구조를 통해 성능 향상
- ResNet (2015) : Residual Block으로 기울기 소실 문제를 해결
- Vision Transformer (2020): CNN을 넘어 Transformer 기반의 비전 모델로 발전

• 최근 연구 동향

- 멀티모달 AI : CLIP, DALL-E와 같은 Vision-Language 모델
- Self-Supervised Learning : 레이블이 없는 데이터에서도 높은 성능을 달성

- 초기 컴퓨터 비전 연구에서 제안된 Hough Transform과 같은 기하학적 알고리즘은 현재도 객체 위치 및 형태 검출을 위한 전통적 전처리 기법으로 계승되어 있음.
- Marr의 다단계 비전 처리(저수준, 중간수준, 고수준) 이론은 오늘날 딥러닝 모델에서도 계층적 특징 추출이라는 개념으로 발전되어 사용되고 있음.
- SIFT, HOG 등 수동 특징 추출 기법과 SVM 기반 분류기는 현대의 딥러닝 모델에서도 데이터 증강이나 초기 특징 표현 방식을 이해하는 기초적 토대로 여전히 영향을 미치고 있음.
- Viola-Jones 얼굴 감지 알고리즘의 AdaBoost 방식은 딥러닝 시대에도 강한 분류기를 만드는 기초 아이디어로서 앙상블 학습 기법과 융합하여 활용되고 있음.
- ImageNet의 대규모 데이터셋 구축과 벤치마크 체계는 현재 컴퓨터 비전 연구의 데이터 중심 접근 방식과 표준화된 평가 방법의 기반이 되어 기술 혁신을 지속적으로 촉진하고 있음.

• 주요응용분야

1.1 자율주행

- **목적:** 자율주행 차량이 주행 환경을 이해하고, 안전한 운행을 계획할 수 있도록 지원합니다.
- **주요 기술:**
 - 객체 감지(Object Detection): 차량, 보행자, 도로 표지판 등을 감지.
 - 세그멘테이션(Segmentation): 도로, 차선, 보행자 구역 등을 분할.
- **사용 사례:**
 - Tesla의 Autopilot: 객체 감지 및 경로 계획에 활용.
 - Waymo의 자율주행차: 완전 자율주행 기술 테스트.

1.2 의료 영상 분석

- **목적:** 의료 이미지를 분석하여 정확한 진단 및 치료 계획 수립을 지원합니다.
- **주요 기술:**
 - 질병 진단: CT, MRI 등 의료 영상을 분석하여 암 및 질병 탐지.
 - 세포 분석: 현미경 이미지를 활용한 세포 형태 및 변이 파악.
- **사용 사례:**
 - 폐 CT 영상에서 결절 탐지.
 - 당뇨망막병증 검출: 안저 이미지를 분석하여 당뇨병 합병증 여부 파악.

1.3 얼굴 인식

- **목적:** 얼굴 데이터를 활용해 인증, 보안, 사용자 맞춤형 서비스를 제공합니다.
- **주요 기술:**
 - 얼굴 검출: 특정 영역에서 얼굴 탐지.
 - 얼굴 임베딩: 얼굴 특징을 벡터 형태로 변환하여 유사도 계산.
- **사용 사례:**
 - Apple의 Face ID: 스마트폰 잠금 해제.
 - 공항의 출입 통제 시스템.

1.4 증강현실(AR) 및 가상현실(VR)

- **목적:** 현실과 가상의 경계를 허물고, 몰입형 환경을 제공합니다.
- **주요 기술:**
 - SLAM(Simultaneous Localization and Mapping): 환경 스캔 및 객체 배치.
 - 3D Reconstruction: 현실 공간의 3D 모델 생성.
- **사용 사례:**
 - Pokémon GO: 현실 세계에서 증강된 가상 캐릭터 배치.
 - Meta Oculus: 몰입형 VR 경험 제공.

1.5 멀티모달 AI와 Vision-Language 모델

- 목적: 이미지를 텍스트와 결합하여 풍부한 이해와 표현을 가능하게 합니다.
- 주요 기술:
 - CLIP: 이미지와 텍스트를 동시에 학습하여 연관성을 이해.
 - DALL-E: 텍스트를 기반으로 이미지를 생성.
- 사용 사례:
 - AI 기반 이미지 캡션: 이미지를 설명하는 텍스트 생성.
 - 텍스트에서 이미지 생성: 광고 및 콘텐츠 제작.

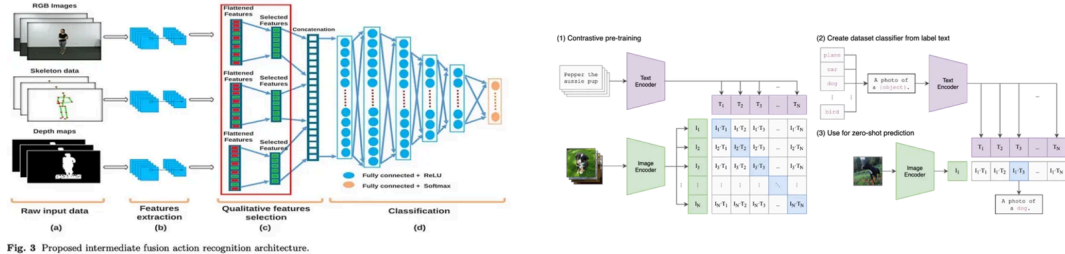


Fig. 3 Proposed intermediate fusion action recognition architecture.

2 이미지 전처리

2. 이미지 전처리

이미지 전처리는 원본 이미지를 학습 가능한 형태로 변환하는 과정으로, 모델의 성능과 학습 속도를 높이는 데 중요한 역할을 합니다. 이 섹션에서는 이미지 전처리의 필요성과 주요 기법을 다룹니다.

2.1 이미지 전처리의 필요성

1. 데이터의 일관성 확보:

- 데이터 크기, 색상 공간, 해상도의 불균형을 해결.
- 다양한 이미지 데이터를 표준화하여 모델 입력으로 적합한 상태로 변환.

2. 노이즈 제거:

- 센서 결함이나 외부 요인으로 발생한 불필요한 정보를 제거.
- 이미지 품질을 향상시켜 학습 데이터의 신뢰도를 높임.

3. 효율적인 처리:

- 데이터를 간소화하거나 크기를 조정해 처리 속도 향상.
- GPU나 메모리 사용량을 최적화하여 대규모 데이터를 효율적으로 처리.

2.2 주요 전처리 기법

1) 크기 조정 (Resizing)

- 목적:
 - 이미지를 고정된 크기로 변환하여 모든 데이터의 크기를 통일.
 - 예: 224×224, 128×128 크기의 정사각형 형태로 변환.
 - 방법:
 - 비율 유지(aspect ratio)를 고려하거나, 필요한 크기로 비율을 무시하고 조정.
 - 응용:
 - 이미지 분류에서 고정 크기 입력을 요구하는 모델에 사용.
-

2) 색상 공간 변환

- 목적:
 - 이미지 데이터를 다른 색상 공간으로 변환해 특정 특징 추출을 용이하게 함.
 - 방법:
 - RGB → Grayscale: 계산량 감소 및 윤곽 정보 강화.
 - RGB → HSV: 색상(Hue), 채도(Saturation), 명도(Value)를 분리해 객체 구별을 강화.
 - 응용:
 - 얼굴 인식에서는 Grayscale로 전환해 특징 학습 간소화.
 - 객체 탐지에서는 HSV로 전환해 색상 기반 탐지 성능 향상.
-

3) 노이즈 제거

- 목적:
 - 센서 결함, 조명 변화 등으로 인해 이미지에 포함된 불필요한 노이즈를 제거.
 - 방법:
 - Gaussian Blur: 가우시안 필터로 부드러운 노이즈 제거.
 - Median Filter: 주변 픽셀의 중간값을 계산해 잡음을 억제.
 - 응용:
 - 의료 영상에서 작은 노이즈를 제거하여 세포 또는 조직 분석 정확도 향상.
-

4) 히스토그램 평활화 (Histogram Equalization)

- 목적:
 - 이미지의 밝기 분포를 균일하게 만들어 명암 대비를 향상.
 - 방법:
 - 픽셀 값의 빈도수를 분석하여 히스토그램의 분포를 재조정.
 - 응용:
 - 빛이 부족한 저조도 환경에서 촬영된 이미지의 대비를 개선하여 객체를 더 뚜렷하게 나타냄.
-

5) 정규화 (Normalization)

- 목적:
 - 픽셀 값을 일정한 범위(예: 0~1 또는 -1~1)로 변환하여 스케일 차이를 줄임.
 - 방법:
 - 각 픽셀 값을 255로 나눔으로써 0~1 사이로 스케일링.
 - 응용:
 - 딥러닝 모델에서 학습 속도와 안정성을 향상.
-

6) 이미지 자르기 (Cropping)

- 목적:
 - 관심 영역(Region of Interest, ROI)을 추출하여 불필요한 배경 제거.
 - **ROI(Region of Interest)**: 이미지에서 분석 또는 처리하고자 하는 특정 부분을 의미.
 - 방법:
 - 이미지를 슬라이싱하여 필요한 영역만 추출.
 - 응용:
 - 객체 탐지에서 특정 객체를 중심으로 학습 데이터 생성.
-

2.3 이미지 전처리의 실무 활용

1. 이미지 분류:

- 고정 크기 입력과 정규화된 데이터를 통해 모델 학습을 효율적으로 수행.

2. 객체 탐지:

- 노이즈 제거와 ROI 추출로 탐지 성능을 개선.

3. 의료 영상 분석:

- 히스토그램 평활화를 활용하여 저명암 대비 문제를 해결.

2.4 요약

- 이미지 전처리는 데이터의 품질과 일관성을 높이는 과정입니다.
- 주요 기법에는 크기 조정, 색상 공간 변환, 노이즈 제거, 히스토그램 평활화, 정규화, 이미지 자르기(ROI 추출)가 포함됩니다.
- 이러한 전처리 기술은 이미지 데이터를 학습 가능한 상태로 변환하고, 모델 성능 향상에 기여합니다.

3 데이터 증강

3. 데이터 증강 (Data Augmentation)

데이터 증강은 원본 데이터를 다양한 방식으로 변형하여 **새로운 학습 데이터를 생성**하는 기법입니다. 이는 딥러닝 모델의 일반화 성능을 높이고, 데이터 부족 문제를 완화하는 데 중요한 역할을 합니다.

3.1 데이터 증강의 필요성

1. 모델의 과적합(Overfitting) 방지:

- 작은 데이터셋으로 학습하면 모델이 특정 데이터에 과적합될 수 있음.
- 데이터 증강을 통해 모델이 다양한 패턴을 학습하도록 유도.

2. 데이터 다양성 향상:

- 이미지 회전, 크롭, 플립 등 다양한 변환을 통해 새로운 데이터를 생성.
- 현실 세계에서 발생할 수 있는 다양한 조건(조명, 각도, 배경 등)을 시뮬레이션.

3. 데이터 부족 문제 해결:

- 대량의 데이터를 수집하기 어려운 경우, 증강 기법으로 데이터셋 크기를 효과적으로 확장.

3.2 주요 데이터 증강 기법

1) 기하학적 변환 (Geometric Transformations)

- 회전 (Rotation):
 - 이미지를 일정 각도로 회전.
 - 예: -15도, +30도 등 다양한 각도로 회전하여 학습 데이터 다양화.
- 크롭 (Cropping):
 - 이미지의 특정 영역만 잘라 학습.
 - 객체 탐지에서 ROI 영역만 학습하거나 불필요한 배경 제거.
- 플립 (Flipping):
 - 좌우 또는 상하 반전을 통해 이미지의 대칭성을 학습.
 - 예: 좌우 반전으로 새로운 시점의 데이터를 생성.
- 스케일링 (Scaling):
 - 이미지를 확대하거나 축소하여 객체 크기에 대해 모델이 유연하게 대응.

2) 픽셀 기반 변환 (Pixel-level Transformations)

- 밝기 조정 (Brightness Adjustment):
 - 이미지의 전체적인 밝기를 높이거나 낮춤.
 - 어두운 환경이나 조명이 밝은 조건을 시뮬레이션.
- 대비 조정 (Contrast Adjustment):
 - 밝고 어두운 영역 간의 차이를 조정하여 대비를 높이거나 낮춤.
- 색상 조정 (Color Jittering):
 - 색상, 채도, 명도를 무작위로 변형하여 다양한 색상 조건을 학습.
- 노이즈 추가 (Adding Noise):
 - 가우시안 노이즈 등을 추가하여 현실적인 노이즈 조건을 학습.

3) 고급 변환 기법

- **Affine Transformations:**

- 이미지의 크기, 위치, 각도 등을 선형적으로 변환.
- 예: 이동, 비율 조정, 뒤틀림.

- **랜덤 삭제 (Random Erasing):**

- 이미지의 일부 영역을 무작위로 삭제하여 모델이 특정 부분에 의존하지 않도록 학습.

- **Mixup:**

- 두 이미지를 혼합하여 새로운 데이터 생성.

3.3 데이터 증강의 실무 활용

1. 이미지 분류:

- 다양한 각도, 조명 조건에서 동일한 객체를 학습하여 모델의 견고성을 향상.

2. 객체 탐지:

- 스케일 변환과 크롭을 통해 객체의 위치 변화에 대응하는 학습.

3. 의료 영상 분석:

- MRI, CT 데이터에 회전, 밝기 조정 등 적용하여 데이터 다양화.

3.4 데이터 증강 실습

PyTorch를 활용한 데이터 증강 예제

```
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt

# 이미지 로드
image = Image.open("sample.jpg")

# 데이터 증강 파이프라인 정의
transform = transforms.Compose([
    transforms.RandomRotation(30), # 회전
    transforms.RandomHorizontalFlip(), # 좌우 반전
    transforms.RandomCrop(224), # 랜덤 크롭
```

```

    transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5), #
    밝기, 대비, 채도 조정
    transforms.ToTensor() # 텐서 변환
])

# 증강 적용
augmented_image = transform(image)

# 원본 및 증강된 이미지 시각화
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(image)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("Augmented Image")
plt.imshow(augmented_image.permute(1, 2, 0)) # 텐서를 이미지로 변환
plt.axis("off")

plt.show()

```

3.5 요약

- 데이터 증강은 원본 데이터에 다양한 변형을 통해 새로운 데이터를 생성하여 학습 데이터의 다양성을 향상.
- 주요 기법에는 **회전, 크롭, 플립, 밝기 조정, 색상 변환, 노이즈 추가** 등이 포함.
- 데이터 증강은 모델이 다양한 현실 조건에 잘 대응할 수 있도록 일반화 성능을 개선.

4 이미지 처리 파이프라인

4. 이미지 처리 파이프라인

이미지 처리 파이프라인은 원본 이미지를 **모델 학습에 적합한 입력 데이터로 변환**하기 위한 일련의 과정입니다. 이 파이프라인은 전처리, 증강, 배치 생성, 데이터 로딩 등을 포함하며, 대규모 데이터셋을 효율적으로 다루는 데 필수적입니다.

4.1 이미지 처리 파이프라인의 단계

1) 데이터 로드 (Data Loading)

- 정의:
 - 이미지 데이터를 저장 위치에서 읽어와 메모리로 로드.
 - 방법:
 - 파일 시스템에서 이미지를 읽거나, 데이터셋 API(예: PyTorch, TensorFlow)를 활용.
 - 실무 예시:
 - 디스크에서 이미지를 로드하거나, ImageNet, COCO 데이터셋 같은 표준 데이터셋 사용.
-

2) 데이터 전처리 (Preprocessing)

- 정의:
 - 이미지 데이터를 정규화, 크기 조정, 색상 공간 변환 등을 통해 표준화.
 - 주요 작업:
 - 크기 조정: 모델 입력 크기에 맞게 이미지 크기를 통일.
 - 정규화: 픽셀 값을 0~1 또는 -1~1 범위로 변환.
 - 색상 공간 변환: RGB에서 Grayscale 또는 HSV로 변환.
-

3) 데이터 증강 (Augmentation)

- 정의:
 - 데이터를 변형하여 모델이 다양한 조건에 견고하도록 학습.
 - 주요 작업:
 - 회전, 크롭, 플립, 밝기 및 색상 조정.
 - 효과:
 - 데이터 다양성 증가와 과적합 방지.
-

4) 배치 생성 (Batch Creation)

- 정의:

- 데이터를 일정 크기(batch size)로 묶어 모델 학습에 전달.
- **주요 작업:**
 - 데이터셋을 랜덤하게 섞은 뒤(batch shuffle), 배치 단위로 분할.
 - 배치 크기 선택은 메모리 용량과 학습 속도 간 균형을 고려.
- **실무 적용:**
 - GPU를 활용해 병렬 처리 속도 향상.

5) 데이터 로딩 최적화

- **정의:**
 - 대규모 데이터셋을 효율적으로 로딩하여 병목현상 방지.
- **기법:**
 - 멀티스레드 또는 멀티프로세스 로딩.
 - 데이터 캐싱: 자주 사용하는 데이터를 메모리에 유지.
- **도구:**
 - PyTorch `DataLoader`, TensorFlow `tf.data` API.

4.2 실습: PyTorch를 활용한 이미지 처리 파이프라인

코드 예제

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# 데이터 증강 및 전처리 파이프라인 정의
transform = transforms.Compose([
    transforms.Resize((224, 224)), # 크기 조정
    transforms.RandomHorizontalFlip(), # 좌우 반전
    transforms.RandomRotation(15), # 회전
    transforms.ToTensor(), # 텐서 변환
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]) # 정규화
])

# 데이터 로드
```

```
dataset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
```

```
# DataLoader로 배치 생성 및 로딩
```

```
data_loader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=4)
```

```
# 배치에서 데이터 확인
```

```
images, labels = next(iter(data_loader))
```

```
print(f"Batch size: {images.size()}") # 출력: [32, 3, 224, 224]
```

4.3 이미지 처리 파이프라인의 최적화

1. 병렬 처리:

- `num_workers` 파라미터를 사용하여 여러 스레드로 데이터 로딩 속도 향상.

2. 데이터 캐싱:

- 자주 사용하는 데이터는 메모리에 유지하여 디스크 접근 최소화.

3. Prefetching:

- GPU가 계산을 수행하는 동안 데이터를 미리 로드.

4.4 이미지 처리 파이프라인의 실무 활용

1. 대규모 데이터셋:

- ImageNet, COCO와 같은 대규모 데이터셋에서 효율적인 처리 필수.

2. 실시간 처리:

- 자율주행 차량과 같은 애플리케이션에서 빠른 데이터 로딩 및 처리 필수.

3. 모델 학습 및 배포:

- 모델 학습 단계에서는 최적화된 파이프라인으로 속도와 효율성을 극대화.

5 이미지의 통계적 특징

- (참조) 04-02. 정리_이미지처리_필터.ipynb

텍스처 분석

1) 텍스처란?

- 정의: 이미지에서 반복적이고 규칙적인 패턴이나 구조를 의미.
- 예:
 - 나뭇잎의 질감, 천의 조직감, 벽돌의 패턴 등.

2) 텍스처 특징 추출

- 주요 방법:
 - **GLCM (Gray-Level Co-occurrence Matrix):**
 - 픽셀 간의 관계를 기반으로 텍스처를 분석 (두 픽셀이 특정 거리와 방향에서 발생하는 밝기 값 쌍의 빈도를 행렬로 표현)
 - 예: 부드러운 텍스처는 비슷한 밝기 값이 인접해 있음.
 - **Local Binary Patterns (LBP):**
 - 주변 픽셀의 밝기를 기준으로 텍스처 패턴을 추출.
 - 효율적이고 계산량이 적어 실시간 분석에 유리.

- **Local Binary Patterns (LBP):**
 - 주변 픽셀의 밝기를 기준으로 텍스처 패턴을 추출.
 - 효율적이고 계산량이 적어 실시간 분석에 유리.

